Fleet Simulation Engine: Run and Test Manual

This manual covers building, running, and testing the engine. The engine implements Project 2 of the Saronic interview spec: the Base Problem (vessel simulation), Follow-on 1 (cargo and dock operations), and Follow-on 2 (historical state queries). Commands below run from the project root, the folder that holds `Dockerfile`.

This manual is a conversion from *manual.md*. Code lines are wrapped and certain characters are different. Please use the .md file for copying code.

1. Prerequisites

- Docker, to build, run, and test with no local Rust toolchain. Docker (Docker Desktop on macOS and Windows) must be running. The first build downloads the base images and the crates, so it needs network access. The image builds natively on arm64 and amd64; the runtime image is about 40 MB and the test image about 1.6 GB.
- Or a Rust toolchain, version 1.97 or later. `rustup` uses the version pinned in `rust-toolchain.toml` (1.97.0) and installs it if it is missing. The only direct dependencies are `serde` and `serde_json`.

2. Build

```
docker build -t fleet-sim .
```

This compiles the release binary in a Rust build stage and copies it into a minimal runtime image that runs as a non-root user, with the engine as its entrypoint.

Without Docker, `cargo build --release` writes the binary to `target/release/fleet-sim`.

3. Run

The engine follows the spec's I/O model: it reads newline-delimited JSON commands from stdin and writes exactly one JSON line to stdout per command, in the order received. It exits when its input ends.

Each command is one JSON object on one line, `{"command":"<name>","parameters":{...}}`. `parameters` is required, as `{}` when a command takes none. The engine implements all three parts of the spec together, so it uses the Follow-on 1 formats throughout: `init` requires `dock_throughput`, and every world state includes `cargo` lists. The spec's Base Problem `init`, which has no `dock_throughput`, is therefore rejected; section 4 shows that example in the current format.

With a file holding one command per line:

```
docker run -i --rm fleet-sim < commands.jsonl
```

For example, `grep '^{' tests/fixtures/flw1_example.txt > commands.jsonl` writes the commands of the spec's Follow-on 1 example to such a file. `-i` connects stdin to the container; without it the engine receives no input. Without Docker, run `target/release/fleet-sim < commands.jsonl`.

To type commands at the terminal instead:

```
docker run -i --init --rm fleet-sim
```

The engine answers each line as it is entered, until end of input (Ctrl-D). `--init` lets Ctrl-C stop it as well; without it, the engine runs as the container's first process, which ignores Ctrl-C.

Example

```
printf '%s\n' \

'{"command":"init","parameters":{"travel_times":[[0,2],[2,0]],"dock_throughput":[1,1],
"vessels":[{"id":"v1","location":0}],"cargo_manifest":[{"location":0,"destination":1,"
quantity":1}]}}' \
  '{"command":"dock","parameters":{"vessel_id":"v1"}}' \
  '{"command":"tick","parameters":{}}' \
  '{"command":"load","parameters":{"vessel_id":"v1","destination":1}}' \
  '{"command":"load","parameters":{"vessel_id":"v1","destination":1}}' \
  '{"command":"tick","parameters":{}}' \
  '{"command":"get_state_at","parameters":{"tick":1}}' \
  | docker run -i --rm fleet-sim
```

Output:

```
{"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"idle","location":0,"cargo
":[]}},"cargo":[{"location":0,"destination":1,"quantity":1}]}}
{"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"docked","location":0,"car
go":[]}},"cargo":[{"location":0,"destination":1,"quantity":1}]}}
{"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"docked","location":0,"car
go":[]}},"cargo":[{"location":0,"destination":1,"quantity":1}]}}
{"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"docked","location":0,"car
go":[{"destination":1,"quantity":1}]}},"cargo":[]}}
```

```
{"status":"error","payload":{"message":"transfer of 2 operations exceeds dock
throughput of 1 at location 0"}}
{"status":"ok","payload":{"tick":2,"vessels":{"v1":{"state":"docked","location":0,"car
go":[{"destination":1,"quantity":1}]}},"cargo":[]}}
{"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"docked","location":0,"car
go":[]}},"cargo":[{"location":0,"destination":1,"quantity":1}]}}
```


Line by line:

1. `init` returns the world at tick 0: v1 idle at location 0, and one unit for destination 1 in the dock at location 0.
2. `dock` returns the planned state, with v1 docked. The committed state stays idle until the next tick.
3. `tick` commits the dock and advances to tick 1.
4. `load` moves the unit from the dock onto v1 in the planned state.
5. The second `load` is rejected: the dock at location 0 allows one cargo operation per tick (`dock_throughput[0]` is 1). A rejected command leaves the state unchanged.
6. `tick` commits the load and advances to tick 2.
7. `get_state_at` returns the committed state at tick 1, before the load.

Responses

Every success is `{"status":"ok","payload":<world state>}`; every failure is `{"status":"error","payload":{"message":"…"}}` and leaves the state unchanged. A world state has:

- `tick`: the tick of the state.
- `vessels`: an object keyed by vessel id. Each vessel has a `state`, either `idle` or `docked` with a `location`, or `transit` with `from`, `to`, and `arrives_at_tick`, and a `cargo` list of `{destination, quantity}`.
- `cargo`: the dock inventories, one list of `{location, destination, quantity}` for all docks.

Commands

Command	Parameters	Effect
`init`	`travel_times` (N×N, zero diagonal, positive elsewhere), `dock_throughput` (N positive integers), `vessels` (`[{id, location}]`), optional `cargo_manifest` (`[{location, destination, quantity}]`)	Must be the first command, once. Returns the world at tick 0.
`set_destination`	`vessel_id`, `destination`	Plans travel for a vessel that is idle in the committed state. Returns the planned state.
`dock`	`vessel_id`	Plans docking for a vessel that is idle in the committed state, or cancels a planned undock.
`undock`	`vessel_id`	Plans undocking for a docked vessel and drops its pending cargo operations, or cancels a planned dock.
`load`	`vessel_id`, `destination`	Plans moving one unit for `destination` from the dock to a docked vessel.
`unload`	`vessel_id`, `destination`	Plans moving one unit for `destination` off a docked vessel: delivered if the vessel is at `destination`, otherwise added to the dock.
`swap`	`vessel_id`, `load_destination`, `unload_destination`	A load and an unload together, costing one operation.
`tick`	optional `cargo_manifest`	Commits the planned state and advances one tick. Vessels arriving on that tick become idle; manifest cargo is then added to the docks.
`get_state`	`{}`	Returns the planned state.
`get_state_at`	`tick`	Returns the committed state at `tick`, from 0 to the current tick. For the current tick that is the state at the last tick, without changes planned since.

Each dock allows `dock_throughput[location]` cargo operations (load, unload, or swap) per tick, across all vessels docked there.

Choices where the spec is silent

- Blank input lines are skipped; every other line gets exactly one response.
- Unknown fields in a command are rejected rather than ignored.
- `init` needs at least one location, rejects duplicate vessel ids and out-of-range locations, and accepts an empty vessel list. A rejected `init` leaves the engine uninitialized, so a later valid `init` is accepted.
- `set_destination` back to a vessel's committed location is rejected. A planned trip can be redirected, or cancelled by `dock` and then `undock` before the next tick.
- `dock` after a planned `undock` cancels the undock, and `undock` after a planned `dock` cancels the dock, so the vessel keeps its state from the last tick. Cargo operations the undock dropped stay dropped.
- Other redundant dock commands are rejected: `dock` with a dock already planned (`already docked`) or on a docked vessel (`not idle`), and `undock` with an undock already planned or on a vessel that is neither docked nor planned to dock (`not docked`).
- `undock` after `load`, `unload`, or `swap` drops that vessel's pending cargo operations and refunds their throughput. A cargo operation after a planned `undock` replaces it, and the vessel stays docked.
- A load and an unload of the same destination by the same vessel before a tick cancel out, in either order, and the pair costs one operation. `swap` is not paired with other commands.
- Otherwise a cargo operation draws only on the inventory of the last tick: a unit loaded or unloaded since then moves again only after the next tick. Each half of `swap` follows the `load` and `unload` rules, including delivery at the destination.
- `load` checks its destination's range, as `unload` and `swap` do.
- When both checks fail, the throughput error is reported before a missing-cargo error.
- A manifest entry may have its location equal to its destination.
- A manifest that would bring the total cargo units past the integer limit is rejected (`cargo quantity limit reached`).
- The dock `cargo` list keeps each (location, destination) pair where it first appeared, as in the spec examples. An entry that reaches zero is removed, and a later unit for that pair is added at the end. Vessel `cargo` lists work the same way.
- A negative or non-integer `tick` in `get_state_at` is rejected as an invalid command, like any other negative or non-integer number.

4. Replay the spec's examples

`tests/fixtures/` holds the spec's three example transcripts:

File	Spec example
`base_example_flw1.txt`	Base Problem, in the Follow-on 1 response format (adds `dock_throughput` to `init` and empty `cargo` lists), since the engine runs all three parts together
`flw1_example.txt`	Follow-on 1, verbatim
`flw2_example.txt`	Follow-on 2, verbatim; it continues the Follow-on 1 example after tick 13

Each file lists the commands, each followed by a `→` line with the spec's expected response where the spec gives one, and # comments. The ticks the spec skips over ("ticks 5–10 omitted") are inserted as commands with no expected response.

To run an example and compare every response the spec gives, byte for byte, define this shell function (bash or zsh). It returns a failure status on any mismatch, or if the engine does not run:

```
check() {
  local out rc
  out=$(mktemp) || return
  if ! grep -h '^{' "$@" | docker run -i --rm fleet-sim > "$out"; then
    echo "check: the engine did not run (is Docker running, and is fleet-sim built?)"
>&2
    rm -f "$out"
    return 1
  fi
  awk 'FILENAME == ARGV[1] { response[FNR] = $0; lines = FNR; next }
       /^\{/ { n++ }
       /^→ / { sub(/^→ /, ""); compared++
               if ($0 != response[n]) { bad++; print "mismatch at command " n ":\n
expected " $0 "\n  got      " response[n] } }
       END { if (lines + 0 != n + 0) { bad++; print n " commands but " (lines + 0) "
responses" }
             print n " commands, " (compared + 0) " responses compared, " (bad + 0) "
mismatches"
             exit (bad > 0) }' "$out" "$@"
  rc=$?
  rm -f "$out"
  return $rc
}
```

Then:

```
check tests/fixtures/base_example_flw1.txt
check tests/fixtures/flw1_example.txt
check tests/fixtures/flw1_example.txt tests/fixtures/flw2_example.txt
```

Expected output:

```
9 commands, 7 responses compared, 0 mismatches
21 commands, 15 responses compared, 0 mismatches
25 commands, 19 responses compared, 0 mismatches
```

To see the responses themselves, run `grep '^{' tests/fixtures/flw1_example.txt | docker run -i --rm fleet-sim`. Without Docker, replace `docker run -i --rm fleet-sim` with `target/release/fleet-sim`, in the function as well. The test suite replays the same files (section 5), comparing the responses as parsed JSON; `check` compares the exact bytes, which is stricter.

5. Test

With Docker

```
docker build --target test -t fleet-sim-test .
docker run --rm fleet-sim-test
```

The first command compiles the engine and its tests into an image; the second runs every test. Every `test result:` line reads `ok` and `0 failed`. In this version the counts are 147 unit tests, 24 in `tests/base.rs`, 27 in `tests/flw1.rs`, and 11 in `tests/flw2.rs` (209 in total).

To run some of the tests, pass a name filter:

```
docker run --rm fleet-sim-test cargo test --locked --offline validation_
```

With a local Rust toolchain

```
cargo test
cargo test validation_
```

What the tests cover

- Unit tests sit beside the code in `src/` and check the rules of each module.
- Black-box tests in `tests/` run the built binary over stdin and stdout, as the spec's I/O model describes: `tests/base.rs` for the Base Problem, `tests/flw1.rs` for Follow-on 1, and `tests/flw2.rs` for Follow-on 2.
- Each requirement test is named with the ID of the design document requirement it verifies, e.g. `base_3_02_…` for BASE-3-02, so `cargo test base_3_02` runs the tests named for that requirement.
- Tests named `validation_…` check the spec's examples: three replay the transcripts in `tests/fixtures/` (section 4), and the others check its shorter per-command examples, such as the documented failure messages.

`cargo test -- --list` lists every test by name, or `docker run --rm fleet-sim-test cargo test --locked --offline -- --list` with Docker.